

Kiro: An Agentic, Spec-Driven Integrated Development Environment

A Technical Review of Architecture, Core Capabilities, and Security Model

Compiled from the official Kiro documentation (kiro.dev/docs) — July 2026

Abstract

Kiro is an agentic integrated development environment (IDE), built on a Visual Studio Code-compatible foundation, developed by the Kiro team under Amazon Web Services (AWS). Kiro distinguishes itself from conventional AI-assisted coding tools through a structured, artifact-based development methodology referred to as “spec-driven development,” complemented by an event-driven automation layer (Agent Hooks), persistent contextual guidance mechanisms (Steering), native support for the Model Context Protocol (MCP), and a dual-mode execution model (Autopilot and Supervised) governing the degree of human oversight over autonomous agent actions. This paper presents a systematic technical review of Kiro's documented capabilities, examining its architecture, operational workflows, configuration schemas, and security posture. The analysis is derived exclusively from Kiro's official documentation portal and is intended to serve as a consolidated technical reference for researchers, engineering teams, and practitioners evaluating agentic IDEs for professional software development.

Keywords: *agentic IDE, spec-driven development, large language models, Model Context Protocol, autonomous software agents, developer tooling, AI pair programming, software engineering automation.*

1. Introduction

The emergence of large language models (LLMs) capable of reasoning over source code has catalyzed a new category of developer tooling known as agentic IDEs — environments in which an AI agent does not merely suggest code completions but actively plans, executes, and verifies multi-step engineering tasks. Kiro positions itself within this category with the explicit design goal of helping developers “do their best work” by combining conversational assistance with formalized planning artifacts.

Unlike purely conversational “vibe coding” tools, Kiro introduces a dual paradigm in which developers can choose between free-form, exploratory interaction (Vibe) and a structured, traceable planning process (Specs). This duality is central to Kiro's value proposition: it allows rapid prototyping while still offering the rigor required for production-grade feature development, team collaboration, and auditable engineering decisions.

This paper synthesizes the documented technical capabilities of Kiro across six domains: (i) structured specifications (Specs), (ii) event-driven automation (Hooks), (iii) the conversational agent interface (Chat), (iv) persistent contextual guidance (Steering), (v) external tool integration via the Model Context Protocol (MCP), and (vi) the privacy, security, and governance model underpinning autonomous agent operation. Section 2 describes the methodology used to compile this review. Sections 3 through 8 detail each capability domain. Section 9 discusses installation and platform support, and Section 10 offers concluding remarks.

2. Methodology

The information presented in this document was extracted directly from Kiro's official documentation site (<https://kiro.dev/docs/>) as retrieved in July 2026. The review covers the primary landing page of the documentation and its principal sub-sections: Get Started (installation and first project), Specs, Hooks, Chat, Steering, Model Context Protocol (MCP), and Privacy & Security. Content has been paraphrased, restructured, and synthesized into an academic register; tables reproduce structured reference data (such as configuration fields and trigger types) in a condensed form for technical clarity. No claims beyond what is documented are introduced; where the

documentation references further sub-pages not directly reviewed (e.g., detailed Hook Actions or MCP configuration guides), this is noted explicitly rather than inferred.

3. System Overview and Core Capability Set

Kiro organizes its functionality around six interconnected capability domains, summarized in Table 1. Each is described in detail in the sections that follow.

Capability	Function
Specs	Structured artifacts (requirements, design, tasks) that formalize feature and bug-fix development.
Hooks	Event-driven automations triggered by IDE and agent lifecycle events.
Agentic Chat	Natural-language conversational interface with codebase-aware context and intent detection.
Steering	Persistent, file-based contextual guidance (workspace, global, and team-level).
MCP Servers	Protocol-based integration of external tools, data sources, and prompt templates.
Privacy & Security	Governance model comprising Autopilot/Supervised modes, protected paths, and trusted commands.

Table 1. Summary of Kiro's core capability domains.

4. Structured Specifications (Specs)

Specs are structured artifacts that formalize the development process for features and bug fixes. They provide a systematic approach for transforming high-level ideas into detailed implementation plans with explicit tracking and accountability. According to the documentation, specs enable teams to break down requirements into user stories with acceptance criteria, build design documentation including sequence diagrams and architecture plans, track implementation progress across discrete tasks, and improve collaboration between product and engineering functions.

4.1 Core File Structure

Every spec generates three canonical files:

- requirements.md (or bugfix.md) — captures user stories and acceptance criteria, or structured bug analysis.
- design.md — documents technical architecture, sequence diagrams, and implementation considerations.
- tasks.md — provides a detailed, discrete, and trackable implementation plan.

4.2 Three-Phase Workflow

All specs proceed through three sequential phases. The Requirements/Bug Analysis phase defines what must be built or fixed — for feature specs this yields user stories and acceptance criteria, while bugfix specs capture current, expected, and unchanged behavior. The Design phase produces system architecture, component design, sequence diagrams, data flow, and error-handling and testing strategy. The Tasks phase generates discrete, executable, and independently trackable implementation tasks that report real-time status as work proceeds.

4.3 Task Execution and Parallelism

Kiro exposes a task-execution interface for tasks.md that reports real-time status (in-progress or completed). When a user invokes “Run all Tasks,” Kiro constructs a dependency graph over the task list and groups independent tasks into execution “waves.” Wave 1 comprises all tasks without dependencies and executes concurrently; Wave 2

comprises tasks whose dependencies were satisfied by Wave 1; this continues until all tasks complete. Waves execute sequentially with respect to one another, while tasks within a given wave execute concurrently — a design intended to reduce total execution time for well-decomposed feature specs without additional developer configuration.

4.4 Spec Typology

Kiro supports two categories of specs. Feature Specs guide requirements gathering, technical design, and implementation planning, offering Requirements-First and Design-First workflow variants; a “Quick Plan” mode allows well-understood features to auto-generate all three artifacts without intermediate approval gates. Bugfix Specs are oriented toward systematic root-cause diagnosis, fix design, and regression validation.

4.5 Specs versus Vibe Coding

The documentation recommends Specs for complex features requiring structured planning, bug fixes where regressions carry high cost, work requiring durable documentation for team collaboration, and situations in which requirements or designs require iteration. Conversely, unstructured “Vibe” interaction is recommended for quick exploratory coding and prototyping without well-defined goals.

5. Agent Hooks: Event-Driven Automation

Agent Hooks configure automated responses to IDE and agent lifecycle events, including file save, creation, or deletion; user prompt submission and agent turn completion; and pre- or post-tool invocation, including around spec task execution. Hooks are intended to maintain consistent code quality, prevent security vulnerabilities, reduce manual overhead, and standardize team processes.

5.1 Operating Model

The hook system follows a two-step process: event detection, in which the system monitors for specific IDE or agent events, and automated action, in which a triggered event executes either an agent prompt or a shell command.

5.2 Configuration Schema

Hooks are defined as JSON files stored at `.kiro/hooks/` within the workspace. The configuration schema is summarized in Table 2.

Field	Type	Required	Description
name	string	Yes	Identifier shown in telemetry.
description	string	No	Documentation only.
trigger	string	Yes	The event that fires the hook.
matcher	regex	No	Filters by tool name or file path (default: always-match).
action	object	Yes	{type: "command", command} or {type: "agent", prompt}.
timeout	integer (s)	No	Default 60; 0 disables; ignored for agent actions.
enabled	boolean	No	Default true; false disables without deletion.

Table 2. Hook JSON configuration fields.

5.3 Trigger Reference

Trigger	Fires when	Blocking?
---------	------------	-----------

SessionStart	A session begins.	No
Stop	The agent completes its turn.	No
PreToolUse	Before a tool executes (matches tool name).	Yes
PostToolUse	After a tool executes (matches tool name).	No
PreTaskExec	Before a spec task starts.	Yes
PostTaskExec	After a spec task finishes.	No
UserPromptSubmit	A user submits a prompt.	Yes
PostFileCreate	After the agent creates a file (matches path).	No
PostFileSave	After the agent saves a file (matches path).	No
PostFileDelete	After the agent deletes a file (matches path).	No

Table 3. Hook trigger types and blocking behavior.

5.4 Exit Code Semantics

For command-type actions, exit code 0 signals success, with STDOUT added to context for SessionStart and UserPromptSubmit triggers (and otherwise ignored); exit code 2 blocks execution for blocking-capable triggers (PreToolUse, UserPromptSubmit, PreTaskExec) and returns STDERR to the agent; any other exit code produces a warning to the user while tool execution proceeds unimpeded.

5.5 Hook Creation Modalities

Hooks may be authored via natural-language description (Kiro drafts the configuration), a manual form-based editor exposing fields for title, description, event type, tool-name or file-pattern matchers, and action type, or direct JSON authoring. The Hook UI is reachable via the Command Palette (“Kiro: Open Kiro Hook UI”). The documentation notes that manual hooks, in the legacy sense, have been superseded by manual steering files.

6. Agentic Chat Interface

The Chat module is Kiro's primary conversational surface, providing contextual understanding of the codebase, smart intent detection distinguishing informational from action-oriented requests, session management with persistent history, and support for long-running background processes (“Dev Servers”).

6.1 Access and Multi-Language Support

Chat is accessible via keyboard shortcut (Cmd+L / Ctrl+L), the Command Palette, or a secondary side-bar toggle (Cmd+Opt+B / Ctrl+Alt+B). Kiro chat supports natural-language conversation in multiple languages — documented examples include Mandarin, French, German, Italian, Japanese, Spanish, Korean, Hindi, and Portuguese — automatically detecting the input language and responding in kind, a capability attributed to the underlying LLM rather than a dedicated translation layer.

6.2 Smart Intent Detection

Kiro analyzes user messages to distinguish informational queries (e.g., “How does this work?”), which are answered with explanation and documentation without modifying code, from directive requests (e.g., “Create a component,” “Fix this bug”), which trigger proposed or executed code changes, command execution, or file management — without requiring the user to invoke explicit mode-switching commands.

6.3 Context Providers

Kiro automatically analyzes open editor files, their dependencies, and structure, and additionally exposes an explicit context-injection mechanism via the “#” symbol. Documented context providers are summarized in Table 4; multiple providers can be composed within a single message.

Provider	Description
#codebase	Automatically locates relevant files across the project.
#file	References a specific file.
#folder	References a folder and its contents.
#git diff	References current Git changes.
#terminal	Includes recent terminal output and command history.
#problems	Includes all diagnostics/problems in the current file.
#url	Includes fetched web documentation content.
#code	Includes an inline code snippet.
#repository	Includes a structural map of the repository.
#current	References the active editor file.
#steering	Includes a specific steering file.
#docs	References documentation files/content.
#spec	Includes all files of a given spec (requirements, design, tasks).
#mcp	Accesses MCP tools, prompts, and resource templates.

Table 4. Chat context providers.

6.4 Sessions, History, and Export

Kiro maintains conversation history within sessions, supports creation of new sessions and switching between them, and preserves a detailed execution history of actions such as code changes, executed commands, search results, and file operations, which can be searched, restored, or deleted. Conversations can be exported to Markdown format via the tab context menu.

7. Steering: Persistent Contextual Guidance

Steering provides Kiro with persistent, file-based knowledge about a workspace, obviating the need to restate conventions in every conversation. According to the documentation, steering yields consistent code generation aligned with established patterns, reduced repetition of workspace standards, team alignment across contributors, and scalable project knowledge that evolves alongside the codebase.

7.1 Scope: Workspace, Global, and Team

Workspace steering files reside at .kiro/steering/ within the workspace root and apply only to that workspace. Global steering files reside at ~/.kiro/steering/ and apply across all workspaces; in the event of conflicting instructions, workspace-level steering takes precedence over global steering. Team steering extends the global mechanism by allowing centrally defined files to be distributed via MDM solutions, Group Policy, or a shared repository, then placed into each user's global steering directory.

7.2 Foundational Steering Files

Kiro can auto-generate three foundational files that are included in every interaction by default: product.md (product purpose, target users, key features, and business objectives), tech.md (frameworks, libraries, tools, and

technical constraints, which Kiro prefers over alternatives when generating implementations), and `structure.md` (file organization, naming conventions, import patterns, and architectural decisions).

7.3 AGENTS.md Support

Kiro supports the community AGENTS.md standard as an alternative steering mechanism. AGENTS.md files are markdown-formatted like native steering files but do not support inclusion modes and are always included; they may be placed at the global steering location or the workspace root.

7.4 Inclusion Modes

Steering files declare an inclusion mode via YAML front matter, which must appear as the very first content in the file. Four modes are documented, summarized in Table 5.

Mode	Configuration	Behavior
always (default)	inclusion: always	Loaded into every interaction; suited to universal standards.
fileMatch	inclusion: fileMatch; fileMatchPattern	Loaded only when the active file matches a glob pattern (single or array).
manual	inclusion: manual	Loaded on demand via #file-name mention or slash command.
auto	inclusion: auto; name; description	Loaded automatically when the request matches the file's description, analogous to Skills.

Table 5. Steering file inclusion modes.

7.5 File References and Best Practices

Steering files can embed live references to workspace files using the syntax `#[[file:<relative_file_name>]]`, keeping guidance synchronized with actual project artifacts (e.g., an OpenAPI specification or a UI component used as a canonical pattern). Documented best practices include keeping each file scoped to a single domain, using descriptive filenames, documenting rationale rather than only prescriptions, providing before/after code examples, excluding secrets and credentials (steering files are part of the codebase), and treating steering changes with the same review rigor as code changes.

8. Model Context Protocol (MCP) Integration

MCP is a protocol enabling Kiro to communicate with external servers exposing specialized tools, prompts, and resources — for example, an AWS Documentation MCP server that allows searching, reading, and receiving recommendations from AWS documentation directly within the IDE. Through MCP, Kiro can access specialized knowledge bases, integrate with external services and APIs, extend its tool set with domain-specific capabilities, use server-provided prompt and resource templates via the “#” mention system, respond to server elicitation requests for additional input during tool execution, and support custom tools tailored to specific workflows.

8.1 Setup and Management

Using MCP requires the latest version of Kiro and satisfying any prerequisites documented by individual MCP servers. Configured servers are enabled via Settings (Cmd+, / Ctrl+,) by searching for and enabling MCP support. An MCP Servers tab within the Kiro panel lists configured servers, their connection status, and provides quick access to invoke individual tools, inserting a placeholder prompt into chat.

8.2 Troubleshooting

Diagnostic logs are accessible via the Kiro panel's Output tab under “Kiro - MCP Logs.” Documented common issues include connection failures (verify prerequisites), permission errors (verify token/API key validity), unresponsive tools (review MCP logs), and configuration not loading (validate JSON syntax).

9. Privacy, Security, and Governance Model

Kiro's security model centers on the distinction between two interaction modes — Autopilot and Supervised — that govern the review workflow for agent-generated changes, without altering the underlying permissions or access scope granted to the agent. The documentation explicitly cautions that Supervised mode is a code-review workflow, not a security control: it does not function as a sandbox, isolation boundary, or access-control mechanism.

9.1 Autopilot versus Supervised Mode

Capability	Autopilot	Supervised
Agent can read/write files	Yes	Yes
Agent can run commands	Yes (trusted-commands policy)	Yes (trusted-commands policy)
Review before changes applied	No — applied immediately	Yes — diff reviewed, accept/reject
Revert capability	Manual (revert / checkpoints)	Automatic on rejection, plus manual
Protected-path enforcement	Yes	Yes

Table 6. Comparison of Autopilot and Supervised interaction modes.

In Autopilot mode (the default), Kiro executes multiple steps without per-step approval, applying changes immediately; the user can interrupt at any time and revert via “Revert all changes” or checkpoints. In Supervised mode, the agent pauses after modifying files and presents a diff for review at hunk-level granularity; rejection triggers automatic reversion to the pre-turn state, and consecutive edits to the same file may be batched to reduce interruption. Regardless of mode, shell commands follow a separate approval path: by default, all commands require explicit approval unless pre-authorized via the trusted-commands mechanism.

9.2 Trusted Commands

Kiro requires approval before executing any command unless it matches a user-defined trusted-commands list, configured via “Kiro Agent: Trusted Commands” in settings. Matching uses simple string-prefix logic: exact matches (e.g., npm install), wildcard matches (e.g., npm * trusts all npm subcommands), or a universal wildcard (* alone, trusting all commands, which the documentation explicitly recommends using with extreme caution). The system does not parse command structure, shell chaining, or special characters — responsibility for safe pattern configuration rests entirely with the user.

9.3 Protected Paths

Certain workspace paths require explicit approval before being written, regardless of interaction mode, to prevent unintended modification of sensitive configuration. Documented protected patterns include .vscode/, vscode~, .git/, git~, .code-workspace (path-contains matches), and the exact basenames .git, mcp.json, and .kiroignore. A path-contains match triggers on any occurrence of the pattern within the file path, whereas an exact-basename match requires the final path segment to equal the pattern precisely.

9.4 Recommended Security Practices

The documentation notes that, when authenticated via GitHub or Google, the Kiro agent operates within the user's local environment and may access local files and repositories, environment variables, AWS credentials, and other sensitive configuration files. Recommended mitigations include workspace isolation (segregating sensitive projects, using .gitignore, leveraging IDE workspace-trust features), operating within a clean or containerized environment scoped to the current project's resources, careful management of AWS credentials (temporary credentials, named profiles, removal when not actively needed), and repository access control (reviewing GitHub-accessible repositories, using repository-scoped tokens, auditing permissions regularly). The documentation frames these as general guidelines rather than a complete security solution.

9.5 Remote Extensions

Kiro supports Open VSX extensions, including community-maintained remote SSH extensions, to preserve a familiar development experience. The documentation warns that remote extensions open a connection between the local and remote machine, and that a compromised remote could leverage this connection to execute code locally; users are advised to connect only to remote machines they trust and own, and are reminded that third-party extensions are not developed, maintained, or supported by Kiro.

10. Installation and Platform Support

Kiro is distributed for macOS (Intel and Apple silicon), Windows 10/11 (64-bit only; ARM is not currently supported), and Linux (requiring glibc 2.39 or higher — e.g., Ubuntu 24+, Debian 13+, Fedora 40+, Arch Linux, or Linux Mint 22+). Installation follows a standard download-and-run flow from kiro.dev. On first run, users authenticate via a social or AWS identity provider, optionally import Visual Studio Code settings and extensions, select a theme, and enable shell integration so the agent can execute commands on the user's behalf. Users may downgrade to a previous release from the [downloads page](#); settings, extensions, and sign-in state are preserved across reinstallation. Documented first-class language guides cover TypeScript/JavaScript, Java, and Python.

11. Discussion

Considered as an integrated whole, Kiro's documented architecture reflects a design philosophy in which autonomy and structure are treated as complementary rather than opposing forces. Specs impose process rigor on complex or high-stakes work while Vibe/Chat preserves low-friction exploration; Hooks externalize repetitive quality and security enforcement as declarative, event-driven configuration; Steering decouples organizational knowledge from any single conversation, making agent behavior reproducible across sessions and team members; and MCP provides an extensible boundary through which Kiro can be integrated into heterogeneous toolchains without bespoke, one-off integrations.

The security model is notable for its explicit disclaimer that Supervised mode is a review workflow rather than an isolation mechanism — an important distinction for organizations evaluating Kiro for use in sensitive codebases, since the meaningful security boundaries are the protected-paths list, the trusted-commands policy, and external practices such as credential scoping and workspace isolation, rather than the choice of interaction mode itself. This places the operational burden of access control substantially on the adopting organization's configuration discipline.

Taken together, the reviewed documentation positions Kiro as a tool oriented toward professional, team-based software engineering workflows, with particular emphasis on traceability (through spec artifacts and execution history), extensibility (through MCP and custom hooks), and configurable autonomy (through Autopilot/Supervised modes and steering inclusion policies).

12. Conclusion

This paper has provided a structured technical synthesis of Kiro's documented capabilities as an agentic IDE: spec-driven development for traceable feature and bug-fix planning; event-driven hooks for workflow automation; a context-aware, multilingual conversational agent; persistent, scoped steering for organizational knowledge; MCP-based external tool integration; and a dual-mode governance model balancing autonomous execution against human oversight. Each capability is underpinned by explicit configuration schemas (JSON hook definitions, YAML steering front matter) that render Kiro's behavior auditable and version-controllable alongside application source code. Future documentation review should examine the referenced but not yet analyzed sub-pages — including detailed Hook Actions, MCP server configuration, Agent Skills, and Focus Mode — to extend this technical picture as Kiro's feature set continues to evolve.

References

Kiro Documentation Portal. (2026). Get started. Retrieved from <https://kiro.dev/docs/>

Kiro Documentation Portal. (2026). Specs. Retrieved from <https://kiro.dev/docs/specs/>

Kiro Documentation Portal. (2026). Hooks. Retrieved from <https://kiro.dev/docs/hooks/>

Kiro Documentation Portal. (2026). Chat. Retrieved from <https://kiro.dev/docs/chat/>

Kiro Documentation Portal. (2026). Steering. Retrieved from <https://kiro.dev/docs/steering/>

Kiro Documentation Portal. (2026). Model Context Protocol (MCP). Retrieved from <https://kiro.dev/docs/mcp/>

Kiro Documentation Portal. (2026). Privacy and security. Retrieved from <https://kiro.dev/docs/privacy-and-security/>

Kiro Documentation Portal. (2026). Installation. Retrieved from <https://kiro.dev/docs/getting-started/installation/>